The Art of Computer Programming 3

Lecture Notes

Lecture 1.	**Computable and uncomputable functions**	**2**
Lecture 2.	**Decidable and enumerable languages**	**6**
Lecture 3.	**Turing machines**	**12**
Lecture 4.	**A brief history of computer programming**	**—**
Lecture 5.	**Lambda calculus**	**20**
Lecture 6.	**More lambda calculus**	**26**
Lecture 7.	**Time complexity, classes P and EXP**	**30**
Lecture 8.	**Class NP**	**34**

Lecture 1. Computable and uncomputable functions

1.1 Alphabets and words

At the low level, information inside computers is represented by 0s and 1s. Every program takes as input a finite sequence of bits and returns another such sequence, and all computation as we know it, in the end, is manipulation of bit-strings. Our formalisation begins here.

Definition 1 (Alphabets and words)**.** *An **alphabet** Σ is a non-empty finite set of symbols. A **word** (or **string**) over Σ is a finite sequence of symbols $w = a_1 a_2 \dots a_n$, where each $a_i \in \Sigma$. The unique sequence of length zero is the **empty word**, denoted ε. The set of all finite words over Σ is denoted Σ^*.*

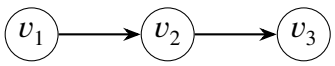
Given two words $u = a_1 \dots a_n$ and $v = b_1 \dots b_m$, their **concatenation** is $uv = a_1 \dots a_n b_1 \dots b_m$. Concatenation is associative: $(uv)w = u(vw)$, and ε is the two-sided identity: $u\varepsilon = \varepsilon u = u$.[1]

Binary encoding. The 'canonical' alphabet for computers is the *binary alphabet* $\{0, 1\}$. Words in any other alphabet Σ can be *encoded* by binary words as follows. We fix a length k such that $2^k \geq |\Sigma|$, and assign a distinct k-bit binary word to each symbol a_i. For example, if $\Sigma = \{a, b, c\}$, we can choose $k = 2$ and map

$$a \mapsto 00, \quad b \mapsto 01, \quad c \mapsto 10.$$

A word over Σ is then encoded by replacing each symbol with its code: for instance, cab becomes 100001. Since every symbol uses exactly 2 bits (or, in general, k), decoding is unambiguous: we simply read the string two bits at a time.[2]

Other finite objects that programs work with are encoded similarly. For example, a directed graph G on vertices $\{v_1, \dots, v_n\}$ can be represented by its $n \times n$ adjacency matrix A, where $A_{i,j} = 1$ if there is an edge $v_i \to v_j$ and 0 otherwise. Flattening this matrix row by row yields a binary word of length n^2. For instance, the graph on 3 vertices with edges $v_1 \to v_2$ and $v_2 \to v_3$ and its corresponding adjacency matrix are:



$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix},$$

which is encoded by the word 010001000.

1.2 Computable functions

Fix your favourite general-purpose programming language (we will use Python in these notes). Consider programs that take one input word and produce one output word over some alphabet Σ. As explained above, multiple arguments, numbers, graphs, or structured data can always be encoded into a single word in Σ^* (and decoded from the output).

Thus, each program π naturally computes a (partial!) function $f_\pi : \Sigma^* \rightharpoonup \Sigma^*$.[3] This is how it is defined.

[1] If you like algebra, you may note that Σ^* with concatenation and ε is thus the free monoid generated by Σ.

[2] In ASCII, a standard encoding in real computers, each character is an 8-bit byte.

[3] Note the arrow! It is $\rightharpoonup$, not $\to$. This is how we denote that the function is *partial*.

Definition 2 (Function computed by a program π)**.** *Given $u \in \Sigma^*$, run π on u.*

- *If π halts (i.e., terminates, concludes the execution), then $f_\pi(u) = v$, where $v \in \Sigma^*$ is the word it outputs.*
- *If π does not halt, then $f_\pi(u)$ is undefined.*

In real life, programs may also crash with an error. In theory, we treat crashes as equivalent to non-halting.

Definition 3 (Computable function)**.** *A partial function $f : \Sigma^* \rightharpoonup \Sigma^*$ is called **computable** if there exists a program π such that $f = f_\pi$.*

Note that when $f : \Sigma^* \to \Sigma^*$ is **total**, i.e., defined on all words in Σ^*, the corresponding program π must halt on every input.

Let us consider some examples of computable functions.

Example 4. The following functions are all computable:

- $reverse$: given $w \in \Sigma^*$, outputs its reverse (e.g., $reverse(abc) = cba$);
- $convert$: interprets $w \in \{0, 1, \ldots, 9\}^*$ as an integer n, outputs a word in $\{0, 1, 2, 3\}$—the same number, but in 4-ary notation;
- $double$: interprets w as an integer n, outputs $2n$;
- $multiply$: given a *pair* of integers $\langle n, k \rangle$, outputs their product $n \cdot k$;
- $shortestPath$: given a directed graph and two vertices $\langle G, s, t \rangle$, outputs the length of the shortest path from s to t (undefined if no path exists);
- $deanon$: given w, if w encodes the real name of Banksy, $deanon(w)$ encodes the real name of Satoshi Nakamoto; for all other w, $deanon$ is undefined.

1.3 Uncomputable functions

Are there any functions that are not computable? Yes, and it is easy to prove: there are only countably many programs, but uncountably many functions on $\{0, 1\}^*$. Let us see this argument in detail.

Recall that a set S is called *countable* if its elements can be indexed by natural numbers. The set $\{0, 1\}^*$

is, of course, countable—here is a possible indexing of words in $\{0,1\}^*$:

$$\begin{aligned} w_0 &= \varepsilon \\ w_1 &= 0 \\ w_2 &= 1 \\ w_3 &= 00 \\ w_4 &= 01 \\ w_5 &= 10 \\ w_6 &= 11 \\ w_7 &= 000 \\ &\dots \end{aligned} \tag{1}$$

That is, we list the words by increasing length, and order words of the same length lexicographically. Similarly, Σ^* is countable for any alphabet Σ.

Programs are words themselves. Indeed, programs are just sequences of symbols that we write, and thus can be seen as words. In fact, the converse is also true: we can try to execute any word as if it was a program and see what happens—a crash would just mean non-halting. So every program is a word and every word encodes some (maybe nonsensical) program. The set of all programs, and hence of all computable functions, is therefore countable.

In contrast, the set of all partial functions $\{0,1\}^* \rightharpoonup \{0,1\}^*$ is uncountable. In fact, this is already true for the set of all functions $\{0,1\}^* \to \{0,1\}$.

Proposition 5 (Cantor, 1891)**.** *The set of all functions* $\{0,1\}^* \to \{0,1\}$ *is uncountable.*

Proof. For clarity of notation, we use the correspondence (1) and consider the set of functions $\mathbb{N} \to \{0,1\}$. Assume for contradiction that this set is countable. Then we can index all functions in this set: $f_0, f_1, f_2, \dots$ Each function f_i is defined by its values on each $k \in \mathbb{N}$. Let us write them out:

$$\begin{array}{lllll} \boldsymbol{f_0(0)} & f_0(1) & f_0(2) & f_0(3) & \dots \\ f_1(0) & \boldsymbol{f_1(1)} & f_1(2) & f_1(3) & \dots \\ f_2(0) & f_2(1) & \boldsymbol{f_2(2)} & f_2(3) & \dots \\ f_3(0) & f_3(1) & f_3(2) & \boldsymbol{f_3(3)} & \dots \\ \dots & & & & \end{array}$$

Consider the diagonal of this infinite table, in bold, and the function:

$$d(n) = \begin{cases} 1, & \text{if } f_n(n) = 0, \\ 0, & \text{if } f_n(n) = 1. \end{cases}$$

Then, d is a function $\mathbb{N} \to \{0,1\}$, so it must appear in the above list $f_0, f_1, f_2, \dots$ of all such functions. However, by construction, d disagrees with f_0 on 0, with f_1 on 1, and, in general, with f_n on n. A contradiction. □

Corollary 6. *There exist uncomputable functions from* $\{0,1\}^*$ *to* $\{0,1\}$.

In fact, there are uncountably many.

And so what? No concrete example was given so far. Could it be that all *useful* functions that we may ever want to compute are countable? Unfortunately, not. Consider the following very useful *halting function*:

$$h(\langle \pi, w \rangle) = \begin{cases} 1, & \text{if } \pi \text{ halts on input } w, \\ 0, & \text{if } \pi \text{ loops forever on input } w. \end{cases}$$

It argument is a *pair*: a program π and a word w, put in the $\langle\rangle$-brackets and separated by a comma—this is just a way to 'pack' two different things into one argument of a function. We assume that $\langle$, $\rangle$ and the comma are all symbols of Σ that are not used in programs and input words.[4] Note that h is a total function.

Theorem 7 (Turing, 1936)**.** *The halting predicate is (alas!) uncomputable.*

Proof. Recall that we have an indexing of words in Σ^*: $w_0, w_1, w_2, \ldots$, each of which can be seen as a program: $\pi_0, \pi_1, \pi_2, \ldots$ Each π_i can (potentially) be run on each w_j (if $i = j$, we run π_i on its own code!)

We can imagine a table of results for the halting predicate:

	w_0	w_1	w_2	w_3	$\ldots$
π_0	$\boldsymbol{h(\pi_0, w_0)}$	$h(\pi_0, w_1)$	$h(\pi_0, w_2)$	$h(\pi_0, w_3)$	$\ldots$
π_1	$h(\pi_1, w_0)$	$\boldsymbol{h(\pi_1, w_1)}$	$h(\pi_1, w_2)$	$h(\pi_1, w_3)$	$\ldots$
π_2	$h(\pi_2, w_0)$	$h(\pi_2, w_1)$	$\boldsymbol{h(\pi_2, w_2)}$	$h(\pi_2, w_3)$	$\ldots$
π_3	$h(\pi_3, w_0)$	$h(\pi_3, w_1)$	$h(\pi_3, w_2)$	$\boldsymbol{h(\pi_3, w_3)}$	$\ldots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

We are going to mess with the diagonal again! Suppose for contradiction that h is computed by some program η. We define the following *diagonal* program δ:

```
def δ(w):
    if η(w, w) == 1:
        while True: pass    # loop forever
    else:
        return 0            # halt immediately
```

Since δ is a valid program, we could ask what happens when δ is run on its own code. If δ halts when run on the word encoding δ, then $\eta(\delta, \delta)$ must return 1, and so δ, according to its own code, should actually loop forever. If it does not halt, $\eta(\delta, \delta)$ returns 0, and then δ should halt. A contradiction. □

[4] This creates a problem: we need to define h also on 'malformed' inputs—such as $\langle\langle, , \rangle$—which is formally a word from Σ^*. But we can agree that h will be equal to 0 on such words and forget about them. What we really care about is the behaviour of h on properly former words: a $\langle$, then a program, then a comma, then an input word, then a $\rangle$.

Lecture 2. Decidable and enumerable languages

2.1 From functions to languages

Let $f : \Sigma^* \to \{0, 1\}$ be a total function that outputs a single bit. It can be thought of as classifying input words into two categories: those on which f returns 1 (the 'yes' words) and those on which f returns 0 (the 'no' words).

This is the same as specifying a subset $L \subseteq \Sigma^*$—namely, the set $L = \{w \in \Sigma^* \mid f(w) = 1\}$.

Conversely, every set $L \subseteq \Sigma^*$ defines a one-bit function, called its **characteristic function**:

$$\chi_L(w) = \begin{cases} 1, & \text{if } w \in L, \\ 0, & \text{if } w \notin L. \end{cases}$$

Every set $L \subseteq \Sigma^*$ is associated with the following problem of binary classification, called the **decision problem**[5] for L:

Input: $w \in \Sigma^*$
Question: Does w belong to L?

Definition 8. *A set of words $L \subseteq \Sigma^*$ is called a **language**. It is **decidable** when its characteristic function is computable, and **undecidable** otherwise.*

A program that computes the characteristic function χ_L is said to **decide** the language L: it outputs 1 on words from L (accepts them) and 0 on words not from L (rejects them). Note that such a program must halt on every input—and give an answer.

We thus have a correspondence between one-bit functions and languages:

	Function $f : \Sigma^* \to \{0, 1\}$	**Language** $L \subseteq \Sigma^*$
Task ('problem')	given w, compute $f(w)$	given w, decide whether $w \in L$
If some π does that	computable	decidable
This π	computes the function	decides the language

Example 9. The empty language $L = \emptyset$ is decidable. It is decided by the following program:

```
def reject_all(w):
    return 0
```

Example 10. Any finite language $L = \{w_0, w_1, \ldots, w_n\}$ is decidable:

[5] The term is historical: basically, we are asked to 'decide' if the input word belongs to the set.

```
def decide_L(w):
    words_L = [w_0, w_1, ... , w_n]
    if w in words_L:
        return 1
    else:
        return 0
```

Example 11. The language

$$\text{PRIMES} = \{w \in \{0,1\}^* \mid w \text{ is the binary encoding of a prime number}\}$$

is decidable:

```
def is_prime(w):
    n = int(w, 2)        # interpret binary string as integer
    if n < 2:
        return 0
    for d in range(2, n):
        if n % d == 0:
            return 0
    return 1
```

Example 12. The language

$$\text{HALT} = \{\langle \pi, w \rangle \mid \pi \text{ halts on input } w\}$$

is undecidable, by Theorem 7.

It is a tradition in the theory of computing to name languages in ALL CAPS. It might be slightly ANNOYING, but that's the tradition, and we will RESPECT IT.

2.2 Semantic properties of programs

We have seen that HALT is undecidable, which is sad for those who would like to *verify* that their programs behave in the expected manner—at least, halt on a given word. Are there properties of programs that one can effectively check?

Of course there are. For example, this one:

Input: program code
Question: Does this program use a variable named x?

It should be clear that the property above is qualitatively different from HALT. The former asks about the program's *syntax*, the latter—about the *function* that the program computes, i.e. the program's *semantics*. To highlight this distinction, we introduce some terminology.

Recall that every $w \in \Sigma^*$ can be treated as a program: feed it to Python's interpreter, specify an input, and it will do something—maybe crash immediately, maybe output Banksy's real name, or maybe output the input backwards...—it will compute some partial function $\Sigma^* \rightharpoonup \Sigma^*$.

Any language $P \subseteq \Sigma^*$ defines a **property of programs**: a program π has property P if $\pi \in P$, and does not have it if $\pi \notin P$.

Definition 13. *A property of programs $P \subseteq \Sigma^*$ is called **semantic** if for any two programs $\pi_1, \pi_2 \in \Sigma^*$, if $f_{\pi_1} = f_{\pi_2}$, then either both $\pi_1, \pi_2 \in P$ or both $\pi_1, \pi_2 \notin P$.*

In other words, a semantic property is a property of *functions*: for every function, either all programs that compute it have the property, or none does.

A property P is **non-trivial** if $P \neq \emptyset$ and $P \neq \Sigma^*$. That is, there are some programs that have it, and some that do not.

Example 14. Consider the property

$$\text{HALT-ON-EMPTY} = \{\pi \mid \pi \text{ halts on input } \varepsilon\}.$$

It is semantic: $\pi \in$ HALT-ON-EMPTY iff $f_\pi(\varepsilon)$ is defined. It is non-trivial: a program that always outputs 1 regardless of the input has this property, and one that loops forever on all inputs does not have it.

The following theorem is the saddest theorem in our course.

Theorem 15 (Rice, 1953)**.** *Every non-trivial semantic property of programs is undecidable.*

Proof. The proof was not covered in the lecture, so it is here just for the curious.

Let $P \subseteq \Sigma^*$ be a non-trivial semantic property, and consider the following program:

```
def never_stop(w):
    while True: pass   # loops forever regardless of the input
```

The function that it computes, $f_{\texttt{never_stop}}$, is the (unique) function that is undefined on all inputs.

Suppose that never_stop $\notin P$. (If never_stop $\in P$, consider the complement $\overline{P} = \Sigma^* \setminus P$ instead, which is also a non-trivial semantic property. Properties P and $\overline{P}$ are either both decidable or both undecidable, so this is without loss of generality.)

Since P is non-empty, there is some program $\sigma \in P$. Since P is semantic and never_stop $\notin P$, we have that $f_\sigma \neq f_{\texttt{never_stop}}$, meaning that σ halts on at least one input.

We reduce the Halting Problem to P. Given a pair $\langle \pi, w \rangle$, construct the following program $\rho_{\pi,w}$:

```
def ρ_{π,w}(x):
    π(w)              # run π on w (ignore x)
    return σ(x)    # if π halted, behave like σ
```

If π halts on w, then $\rho_{\pi,w}$ behaves exactly like σ, so $f_{\rho_{\pi,w}} = f_\sigma$ and $\rho_{\pi,w} \in P$. If π does not halt on w, then $\rho_{\pi,w}$ never reaches the second line, so $f_{\rho_{\pi,w}} = f_{\texttt{never_stop}}$ and $\rho_{\pi,w} \notin P$.

Therefore, $\langle \pi, w \rangle \in \text{HALT}$ if and only if $\rho_{\pi,w} \in P$. If P were decidable by some program `decide_P`, we would be able to decide HALT as follows:

```
def decide_halt(π, w):
    ρ_{π,w} = construct(π, w)    # construct the code of ρ_{π,w} as above
    return decide_P(ρ_{π,w})         # check whether ρ_{π,w} ∈ P
```

This contradicts Theorem 7. □

Example 16. The following languages are all undecidable by Rice's theorem:

- $\{\pi \mid \pi \text{ halts on all inputs}\}$;
- $\{\pi \mid \pi \text{ halts on some input}\}$;
- $\{\pi \mid \pi \text{ accepts the empty word}\}$;
- $\{\pi \mid f_\pi = f_{\pi_0}\}$ for any fixed π_0 — the set of programs equivalent to π_0.

Remark 17. There is one hidden detail in the proof of Rice's theorem—the *construction* of the program $\rho_{\pi,w}$ from π and w. For a proper reduction, we need this construction to be computable: given the code of π and the word w, we must be able to algorithmically produce the code of $\rho_{\pi,w}$. In Python (and any real programming language), this is obvious: $\rho_{\pi,w}$ is just a piece of source code assembled by string concatenation. Formally, a programming language ('encoding') is *admissible* if it has an interpreter and ensures the ability to algorithmically hard-code arguments into a given program's source code. Rice's theorem holds for any admissible encoding, but can fail in artificial, pathological programming languages where programs cannot be algorithmically constructed (imagine, for example, a language where *somehow* every program has an additional flag—whether it halts on the empty input or not).

2.3 Enumerable languages

Definition 18. *A language $L \subseteq \Sigma^*$ is called **recursively enumerable** (or simply **enumerable**) if there exists a program that runs indefinitely and prints a (possibly infinite) list of words, such that:*

(1) every word printed by the program belongs to L;

(2) every word belonging to L is eventually printed by the program (possibly with repetitions).

Example 19. The set $\text{EVEN} = \{n \in \mathbb{N} \mid n = 2k, k \in \mathbb{N}\}$ is enumerable:

```
def enum_even():
    x = 0
    while True:
        print(x)
        x = x + 2
```

Example 20. The language $\{0,1\}^*$ is enumerable:

```
def enum_bin_words():
    words = [""]    # create an array with just ε
    for w in words:
        print(w)
        words.append(w + "0")
        words.append(w + "1")
```

If we replace `print(w)` with `yield w`, `enum_bin_words()` becomes an infinite generator, allowing us to write `for w in enum_bin_words():` to iterate over all words in $\{0,1\}^*$ just like a Python `range`.

By analogy, Σ^* is enumerable and can be iterated through, for any alphabet Σ.

Example 21. The language HALT $= \{\langle \pi, w\rangle \mid \pi \text{ halts on input } w\}$ is enumerable.

Since $\{0,1\}^*$ is enumerable, we can enumerate (binary encodings of) all programs and all input words. But to enumerate the pairs $\langle \pi, w\rangle \in$ HALT, we would need to know if π halts on w. How can we tell, if HALT is undecidable?

Imagine we had infinitely many processors, one for each pair $\langle \pi_i, w_j\rangle$. Then, for every $\langle \pi_i, w_j\rangle$ we can run a process p_{ij}, all of them in parallel, and wait. When a process terminates, we print the respective pair to the output. And those processes that never terminate will never have their pairs printed—that's exactly what we want.

The hard part is how to simulate infinitely many processes on a normal computer. We will rely on the following consideration. At the lowest level, a program does not execute all at once: its execution proceeds as a discrete sequence of elementary *machine instructions*—simple primitive steps: fetching a byte from memory, performing an arithmetic operation, or jumping to another location in code. Each instruction corresponds to a single unit of execution time.

So we can simulate the 'parallelisation' as follows. First, we arrange the infinite two-dimensional grid of processes p_{ij} into a single linear sequence: $p_0, p_1, p_2, \ldots$ (for instance, ordering by the sum $i + j$). Then, we share the processor by gradually allocating more instructions to more processes:

- On step 0, run p_0 for 1 instruction.
- On step 1, run p_0 for a 2nd instruction, and p_1 for 1 instruction.
- On step 2, run p_0 for a 3rd instruction, p_1 for a 2nd instruction, and p_2 for 1 instruction.
- In general, on step s, run p_i for its $(s - i + 1)$-th instruction for each $i \in \{0, 1, \ldots, s\}$.

Our program `enum_halt` will maintain a list of running processes and for each store their current progress. It will iterate through this list, pushing each process forward by one step, and appending newly launched processes. When a process terminates, the respective pair $\langle \pi_i, w_j\rangle$ is printed out.

This technique has a vintage name, *dovetailing*. The metaphor comes from carpentry: a *dovetail joint* joins two boards by interlocking alternating fingers of wood. Here, we similarly interleave small slices of execution from different processes.

Theorem 22 (Post, 1944)**.** *A language $L \subseteq \Sigma^*$ is decidable if and only if both L and its complement $\overline{L} = \Sigma^* \setminus L$ are enumerable.*

Proof. ($\Rightarrow$) Suppose L is decided by a program `decide_L`. Let `enum_words()` be a generator for Σ^*, analogous to the `enum_bin_words` from Example 20. We can enumerate L simply by filtering:

```
def enum_L():
    for w in enum_words():       # iterate over all words in Σ*
        if decide_L(w) == 1:
            print(w)
```

For the complement $\overline{L}$, we do the exact same thing, replacing `== 1` with `== 0`.

($\Leftarrow$) Let `enum_L` and `enum_not_L` enumerate L and $\overline{L}$, respectively. On input w, run `enum_L` and `enum_not_L` in parallel until one of them outputs w (on two processors, or on one, via dovetailing). If `enum_L` outputs w, return 1; if `enum_not_L` outputs w, return 0. Since w must belong to either L or $\overline{L}$ (and never both), exactly one will output w at some point, so this decides L. □

Corollary 23. *The complement of the Halting Problem,*

$$\overline{\text{HALT}} = \{\langle \pi, w \rangle \mid \pi \text{ does not halt on } w\}$$

*is **not** enumerable.*

Lecture 3. Turing machines

3.1 The 1930s: What is computation?

Until now, we have used Python to write programs—our statement of Turing's theorem is that no Python program computes the halting function (in the terminology of Lecture 1) or decides the language HALT (in the terminology of Lecture 2). However, Turing proved his theorem in 1936, before electronic programmable computers existed and when pythons were more dangerous to work with. This deserves an explanation.

The story begins in the 1920s with David Hilbert's grand programme (not program!) to establish the foundations of mathematics. A central question of this programme was the *Entscheidungsproblem* (the 'decision problem'), formulated by Hilbert and Wilhelm Ackermann in 1928:

> *Does there exist an* effective procedure *that takes as input any mathematical statement (encoded in a special notation called 'first-order logic') and decides whether the statement is true or false?*

No electronic computers existed at the time, so it was supposed that such a 'procedure' would be executed by a person with pen and paper. For a procedure to be 'effective', it must be applicable mechanically, by following clear instructions step by step, without creative thinking or other usual attributes of a mathematician's work.

Example 24. Consider solving a quadratic equation $ax^2 + bx + c = 0$, where $a \neq 0$, over the real numbers. Compute the discriminant $\Delta = b^2 - 4ac$. If $\Delta < 0$, there are no real solutions; otherwise, evaluate $x = \frac{-b \pm \sqrt{\Delta}}{2a}$. This is an effective procedure: the steps are explicit and require no mathematical insight or intuition.

Hilbert believed mathematics was decidable: that any well-posed mathematical question could be decided mechanically. To prove this belief rigorously—or to disprove it, as eventually happened—one needed a formal definition of an effective procedure, i.e. of an *algorithm*.

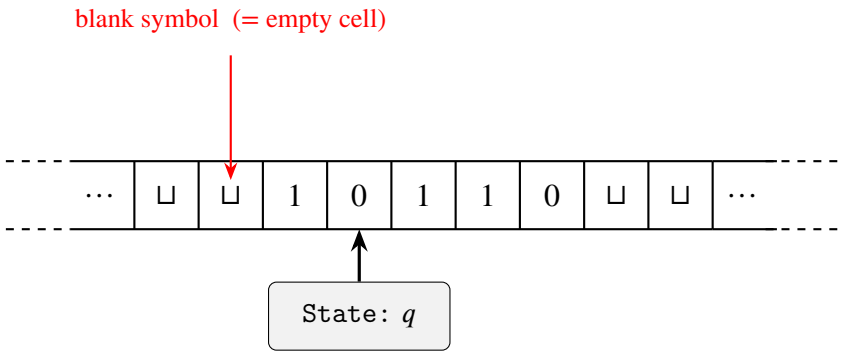
Several definitions of an algorithm were proposed in the 1930s. Although developed by different people, they all turned out to be equivalent in terms of which functions are computable. This is strong evidence that they capture the 'nature of computation'. We will consider some of these definitions in our course.

Probably the most famous definition, and the first that we will study (although not the first historically), was given by Alan Turing in 1936.

3.2 The Turing machine

Informally speaking, a Turing machine is a device consisting of three components:

1. An **infinite tape** divided into discrete cells, extending indefinitely in both directions. Each cell can hold exactly one symbol from a finite **tape alphabet** Γ.
2. A **read/write head** that points to a single cell on the tape at any moment.
3. A **finite control** that stores the internal state q from a finite set of states Q.



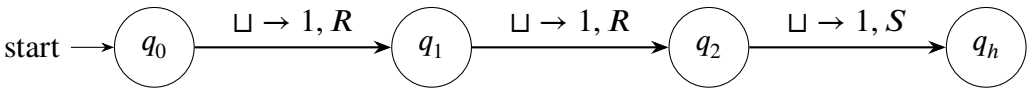
The machine works in steps. At each step, it reads the symbol written in the current cell, and then, depending on that symbol and on the state it is in:

1. overwrites the cell with a new symbol;
2. moves one cell to the right, or to the left, or stays put;
3. switches to another state (of the finitely many available).

A concrete machine is defined by specifying its behaviour for each combination of the observed symbol and current state, and a *halting state*, upon entering which the machine turns off.

Example 25. Consider a machine that, starting on an empty tape (where every cell contains ⊔), prints three 1s and stops.

We can achieve this using four states: q_0 (initial), q_1, q_2, and a halting state q_h. The machine starts in q_0 scanning a blank cell, overwrites it with 1, moves right, and switches to q_1. In q_1, it writes another 1, moves right, and switches to q_2. Finally, in q_2, it writes a third 1 and enters the halting state q_h:



Here, a label $\sqcup \to 1, R$ means "if reading ⊔, write 1 and move right", while S means "stay put".

Formally, we define a Turing machine as follows.

Definition 26. *A **Turing machine** (TM) is a tuple of six components:*

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, q_h \rangle$$

where:

- Q *is a finite, non-empty set of* ***states****;*
- Σ *is an* ***input alphabet****;*
- $\Gamma \supseteq \Sigma$ *is a* ***tape alphabet*** *with a special* ***blank symbol*** $\sqcup \in \Gamma \setminus \Sigma$*;*
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$ *is the* ***transition function****;*
- $q_0 \in Q$ *is the* ***initial state****;*
- $q_h \in Q$ *is the* ***halting state****;*

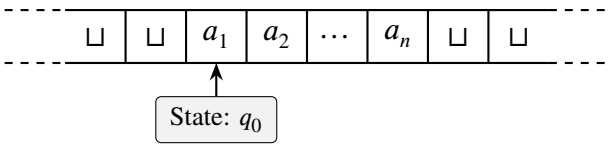
The webpage `https://turingmachine.io/` contains several examples and visualisations of Turing machines, and allows you to program your own. Try it!

3.3 Computing functions and deciding languages on Turing machines

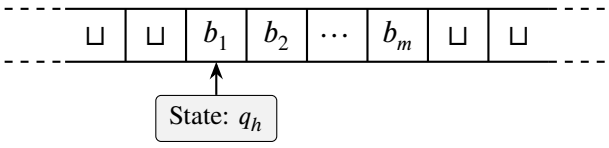
A Turing machine M computes a partial function $f_M : \Sigma^* \rightharpoonup \Sigma^*$, defined as follows:

1. Write the input word $w = a_1 a_2 \dots a_n$ in n consecutive cells, with blanks in all other cells.
2. Place M's head on the first symbol of w (or on a blank cell if $w = \varepsilon$).
3. Start M from its initial state q_0 and let it run:
   - if M ever reaches the halting state (i.e. *halts*), and the resulting tape contains a word $u \in \Sigma^*$ with all other cells blank, then $f_M(w) = u$, provided that the head is observing the first symbol of u when $u \neq \varepsilon$, or a blank cell when $u = \varepsilon$;
   - in all other cases (no halting, symbols not from $\Sigma \cup \{\sqcup\}$ on the tape, more than one word on the tape), $f_M(w)$ is undefined.

Start configuration (input w):



Finish configuration (output $f_M(w) = u$):



Here $u = b_1 b_2 \dots b_m$; when $u = \varepsilon$, the finish configuration contains only blanks.

Clearly, if $\{0, 1\} \subseteq \Sigma$, we can program a machine M computing some function $f_M : \Sigma^* \to \{0, 1\}$. This machine will classify words of Σ^* into two classes and **decide** the language $\{w \in \Sigma^* \mid f_M(w) = 1\}$, as we have seen with programs before.

Decide-by-state version. Consider an equivalent definition of a Turing machine, specifically designed to decide languages. Instead of one halting state it has two: an **accepting state** q_a and a **rejecting state** q_r. Run such a machine on a tape with a word w—if it ever halts, look at the state: q_a means $f_M(w) = 1$, q_r means $f_M(w) = 0$. If it never halts, we will assume that $f_M(w)$ is undefined.

It is widely accepted that Turing machines capture our intuitive notion of an algorithm.

> **The Church–Turing Thesis.** *Every function that can be computed by an effective procedure (in the intuitive sense: by a human following explicit rules, or by any physical computing device) can be computed by a Turing machine.*

This statement is not a mathematical theorem and cannot be proved formally. It says that Turing machines provide a good formal definition of the informal notion of an algorithm.

What gives us confidence in the thesis is the remarkable convergence of different models of computation. In the 1930s, several other logicians independently proposed their own definitions of computability: Alonzo Church formulated the λ-calculus (we will see it in Lecture 5), Kurt Gödel and Jacques Herbrand developed recursive functions, and Emil Post introduced production systems. Despite their very different styles, all of these models turned out to compute exactly the same class of functions. Later formalisations—such as Markov normal algorithms (we will meet them in Exercise Session 4), register machines, and modern programming languages—likewise proved equivalent.

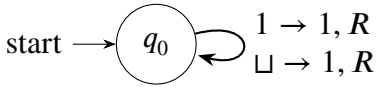
3.4 Examples

As before, we visualise Turing machines as directed graphs where vertices are states and edges represent transitions. A transition $\delta(q, a) = (q', b, D)$ is drawn as an edge from q to q' labelled

$$a \to b, D$$

meaning: "read a, write b, move in direction D".

Example 27. Consider a machine with one work state q_0 over $\Sigma = \{1\}$ and $\Gamma = \{1, \sqcup\}$:



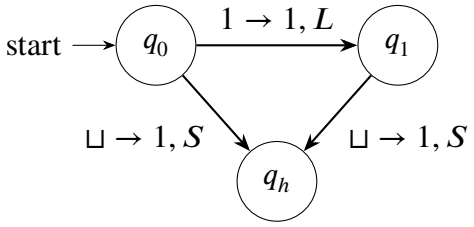
On any input word w, the machine scans rightward indefinitely without ever halting, filling the tape with 1s.[6]

[6] Strictly speaking, we should also specify a halting state q_h—by definition, every machine must have it. But it this machine, it is *unreachable*—no sequence of transitions starting from the initial state ever leads to it, so we omit it from the diagram.

Example 28 (Unary +1)**.** We represent a natural number $n \in \mathbb{N}$ in unary as a sequence of n ones (with 0 represented by the empty word ε).

Given n consecutive 1s on the tape, we wish to produce $n + 1$ consecutive 1s. Rather than walking to the right end of the block, the machine can write 1 to the left of the input: if the head reads 1, it steps one cell left onto the adjacent blank, writes 1, and halts. If the input is empty (the head reads ⊔), it writes 1 in place and halts. In both cases, the machine halts observing the first 1 of the output.

Let $Q = \{q_0, q_1, q_h\}$ with initial state q_0 and halting state q_h:



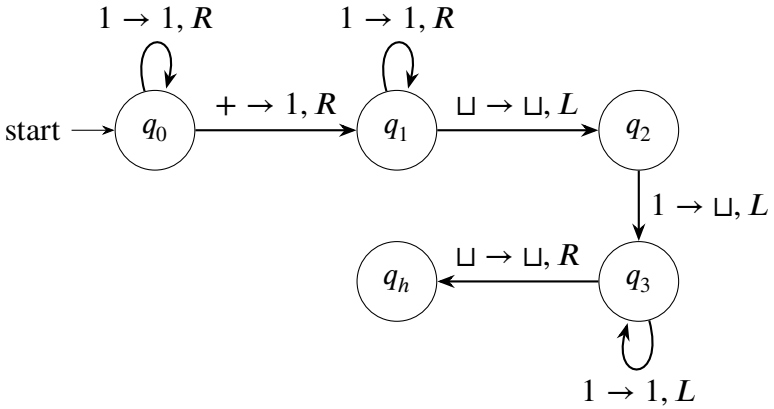
Example 29 (Unary addition)**.** Consider computing the sum of two numbers $m, n \geq 1$ given as two blocks of ones separated by a + symbol:

$$\ldots \sqcup \underbrace{11 \ldots 1}_{m} + \underbrace{11 \ldots 1}_{n} \sqcup \ldots$$

The procedure is as follows:

1. Walk right across the first block of 1s until reaching the + symbol.
2. Replace + with 1, joining the two blocks into a single block of $m + 1 + n$ ones.
3. Walk to the right end of this block, step back onto the rightmost 1, and erase it (write ⊔), leaving $m + n$ ones.
4. Walk left across all 1s until reaching the blank cell ⊔ to the left of the block, then step one cell right onto the first 1 and halt.

Here is the machine $Q = \{q_0, q_1, q_2, q_3, q_h\}$ with initial state q_0 and halting state q_h:

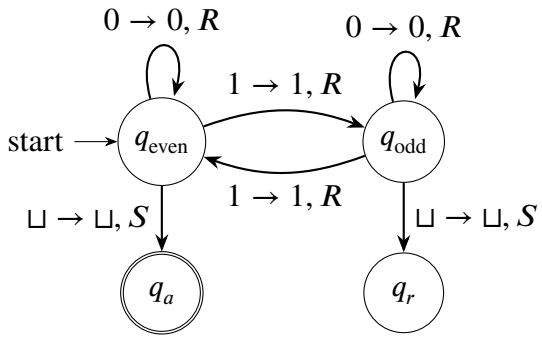


Example 30 (Parity checker)**.** Let us construct a machine with accepting and rejecting states that decides whether a binary string $w \in \{0, 1\}^*$ contains an *even* number of 1s.

We use two work states to track the parity of the number of 1s seen so far: q_{even} (initial state) and q_{odd}. While scanning w, reading 0 keeps the state unchanged, while reading 1 toggles between q_{even} and q_{odd}.

When the machine hits the blank cell $\sqcup$ at the end of the input word:

- from q_{even}, it transitions to the accepting state q_a;
- from q_{odd}, it transitions to the rejecting state q_r.



3.5 The universal Turing machine

A Turing machine is a single-purpose device: its behaviour on every input is fixed by its transition function. One machine computes binary addition; another decides whether a binary word represents an even number. A computer, however, is more versatile: we program it to perform different tasks. This familiar idea was far from obvious in the 1930s, and we owe its mathematical formulation to Alan Turing.

Turing imagined a universal machine that can be programmed to simulate any other Turing machine. Before we define it, let us remove an apparent complication: different Turing machines may have different tape alphabets.

Simulating an alphabet in binary. Let M have tape alphabet Γ. Choose k such that $2^k \geq |\Gamma|$, and assign a different block of k bits to every symbol of Γ. For example, if

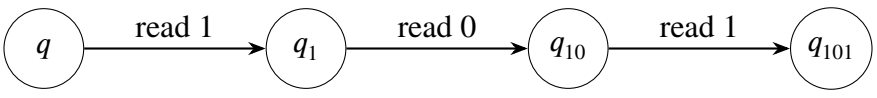
$$\Gamma = \{\sqcup, a, b, c, d, e, 0, 1\},$$

symbol	⊔	a	b	c	d	e	0	1
code	000	001	010	011	100	101	110	111

then three cells of M are represented by three blocks on a binary tape:

a	e	c
0 0 1	1 0 1	0 1 1

The simulating machine reads one block at a time. For every state q of M, it has separate states $q_1, q_{10}, q_{101}, \ldots$ that record both q and the prefix read so far. While reading the block 101, for instance, it follows:



In state q_{101}, the simulator knows both that M is in state q and that the scanned symbol is e. It can therefore apply the transition prescribed for (q, e). Further intermediate states remember the new symbol and direction while the simulator rewrites the three bits and moves to the appropriate neighbouring block.

For k fixed in advance, there are only finitely many positions within a block and finitely many prefixes to remember, so all this information fits into finitely many states. We may therefore replace any fixed tape alphabet by a binary one (note that even the blank symbol $\sqcup$ can be encoded as k consecutive 0s, and thus not needed—an empty tape is just a tape filled with 0s).

Consequently, we may freely use finitely many auxiliary tape symbols. For example, we can mark a symbol a by replacing it with a decorated copy $\hat{a}$, or add separators and flags: these are merely extra symbols, and a larger fixed block size encodes them all in binary.

Encoding a machine. It now suffices to encode machines whose tape alphabet is $\{0, 1\}$, with 0 serving as the blank symbol. Suppose such a machine M has m states. Number them

$$q_0, q_1, \ldots, q_{m-1},$$

renaming if necessary so that q_0 is initial and q_{m-1} is halting. Begin the code with m ones followed by a zero. From this prefix, a decoder learns m and hence the fixed number of bits needed to write any state number.

Next list the two transitions from each state, first the one for reading 0 and then the one for reading 1. A transition record contains the fixed-width binary number of the new state, the bit to write, and two bits for the direction (say, 00 for L, 01 for R, and 10 for S). There are exactly $2m$ such records, all of the same known length. Their concatenation after the prefix is therefore a binary word which unambiguously encodes the machine M. We will denote this word by $[M]$.

The input w is already binary. We encode the pair consisting of M and w, for example, as a word $\langle [M], w \rangle$, where $[M]$ is the binary word above, while $\langle$, $\rangle$ and the comma are special symbols that can also be encoded in binary. The particular convention is unimportant; what matters is that a fixed Turing machine can decode it and determine where $[M]$ ends and w begins.

Simulating an encoded machine. A **universal Turing machine** U receives $\langle M, w \rangle$ and maintains three pieces of information on its tape:

- the description $[M]$;
- the current state of the simulated machine;
- an encoding of its tape, with one marked cell (say $\hat{0}$ instead of 0, or $\hat{1}$ instead of 1) indicating the position of the head of M.

Initially these contain $[M]$, q_0, and the input w. To simulate one step, U reads the marked cell, scans $[M]$ for the transition with the current state and symbol, writes the new symbol, moves the marker to the neighbouring cell, and records the new state. If the marker moves beyond the represented part of the tape, U adds a cell containing the blank symbol 0. Each operation is a finite combination of scans, comparisons, and copies, so a fixed Turing machine can perform it. Repeating this procedure simulates every step of M.

Theorem 31 (Turing, 1936)**.** *There exists a Turing machine U such that, for every Turing machine M and input w,*

$$U(\langle M, w\rangle) = M(w).$$

That is, U halts exactly when M halts, and in that case produces the same output.

The universal machine is the theoretical model of a general-purpose computer: $[M]$ is a program, w is its input, and U executes the program. We can now restate Theorem 7 for Turing machines:

Theorem 32 (Turing, 1936)**.** *The language*

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ halts on } w\}$$

is undecidable.

Lecture 5. Lambda calculus

So far, we have emphasised that programs can be treated as data. Now, we will study another formal definition of an algorithm, historically one of the earliest, where the tables are turned. In Alonzo Church's lambda calculus, everything is a program.

5.1 Functions, in math and in lambda calculus

We are used to defining functions by writing $f(x) = x + 1$, or, sometimes, by $f : x \mapsto x + 1$. Applying f to 4 gives 5: $f(4) = 5$.

In lambda calculus, the same function is denoted by $\lambda x.x + 1$—literally, "a function that takes x and returns $x + 1$".[7] Application goes like this:

$$(\lambda x.x + 1)4 = 4 + 1 = 5$$

This is not pure lambda calculus: to keep this first example simple, we use numbers and the notation $+1$ as if they were defined. Soon we will construct these things, and many more.

A function may return another function. For example,

$$(\lambda x.\lambda y.x + y)3 = \lambda y.3 + y$$

Applying the result to 4 gives 7. Thus, a function of many arguments can be represented by a function receiving the first argument and returning a function of the rest of the arguments, with the first one fixed. This is called **currying**.[8]

Definition 33. *A **lambda term** is a variable x, an application $(M\,N)$ of one term to another, or an abstraction $(\lambda x.M)$.*

Application associates to the left: $M\,N\,P$ means $(M\,N)P$; the parentheses around terms are omitted where unnecessary. The body of an abstraction extends as far to the right as possible: $\lambda x.M\,N$ means $\lambda x.(M\,N)$.

5.2 Computation by substitution

The basic computation rule is **beta reduction**:[9]

$$(\lambda x.M)N \longrightarrow_\beta M[x := N]$$

Read this as: replace the occurrences of the parameter x in the body M by the argument N. It may be performed inside a larger term, including inside the body of an abstraction. We write $\longrightarrow_\beta^*$ for zero or more applications of beta reduction.

[7] Why λ? Church never explained clearly. Wikipedia's page on lambda calculus contains some details.

[8] Chicken curry is not relevant here, Haskell Curry is.

[9] Why β?! Because there is a simpler one, called α, so it's just numbering (and Church's love for Greek letters).

Example 34. Let $\text{ID} = \lambda x.x$ and $\text{K} = \lambda x.\lambda y.x$. Then:

$$\text{ID}A \longrightarrow_\beta A \qquad \text{K}AB \longrightarrow_\beta (\lambda y.A)B \longrightarrow_\beta A$$

The first function returns its argument; the second keeps its first argument and discards the second.

A variable occurrence is **bound** if it is introduced by a surrounding λ; otherwise it is **free**. In $\lambda x.xy$, the occurrence of x is bound and that of y is free. A term with no free variables is **closed**. Substitution replaces only free occurrences: in $\lambda x.x$, the x is bound, it is a local name for $\lambda x.x$.

Python has a similar thing, when one function is defined inside another:

```
def outer(x):
    def inner(x):        # a different parameter
        return x + 10    # inner's x
    return inner(x)      # outer's x

outer(5)  # 15
```

Calling `outer(5)` does not change `inner`'s definition; its parameter receives a value only when `inner(5)` is called. Similarly,

$$(\lambda x.x(\lambda x.x))y \longrightarrow_\beta y(\lambda x.x)$$

The name of a local parameter does not matter: $\lambda x.x$ and $\lambda z.z$ describe the same function. Consistently renaming a bound variable to a fresh name, meaning one not already occurring in the term, is called **alpha conversion**: $\lambda x.x \equiv_\alpha \lambda z.z$. We will identify terms that differ only by such renaming.

Mind the bounds! The term $\text{K} = (\lambda x.\lambda y.x)$ of Example 34 says: "take x, remember it, then take whatever and return x". Imagine we apply it to y: $(\lambda x.\lambda y.x)y$. A 'dumb' reduction substitutes y for x in the body and produces $\lambda y.y$: we were supposed to remember the first argument (which was y), but it is now lost! Indeed, $\lambda y.y$ will take an argument and output *that argument*, not the term y passed as the first argument to K. So this is a wrong way of doing beta reductions. The correct way is: when substituting M for the free occurrences of x in N, first rename any bound parameters of N that would otherwise bind free occurrences of variables in the inserted M, using names fresh for both terms:

$$(\lambda x.\lambda y.x)y \equiv_\alpha (\lambda x.\lambda z.x)y \longrightarrow_\beta \lambda z.y$$

Now the answer is a constant function returning the external y, and we are happy. This convention is called **capture-avoiding substitution**.

Once again, let us consider an example from Python.

```
y = 10

def add_to_y(x):
    return x + y

def calculate(y):
    return add_to_y(3)

calculate(100)  # 13
```

Inside `add_to_y`, `y` refers to the global variable, regardless of where the function is called. Suppose we replace the call by its body, substituting 3 for `x`:

```
def calculate(y):
    return 3 + y

calculate(100)  # 103: wrong!
```

The copied occurrence of `y` now refers to `calculate`'s parameter: it has been accidentally captured. Renaming that parameter before copying the body preserves the meaning:

```
def calculate(t):
    return 3 + y

calculate(100)  # 13
```

Python itself does not substitute text when calling functions, but in λ-calculus, which computes exactly by substitutions, we must use α-conversion to avoid problems as above.

5.3 Functions as arguments of other functions

Function composition is itself a function:

$$\textsc{Comp} = \lambda f.\lambda g.\lambda x.f(gx)$$

What does it do? It takes the first argument, f, then the second, g, and returns a function that takes x and applies g to x, then f to $g(x)$ (in standard math notation), then returns the result $f(g(x))$. Indeed, $\textsc{Comp}FGX \longrightarrow_\beta^* F(GX)$.

Example 35. Let $\textsc{Dbl} = \lambda f.\lambda x.f(fx)$, the operation that applies a function twice. Then

$$\textsc{Dbl}\,FX \longrightarrow_\beta^* F(FX) \qquad \textsc{Dbl}(\textsc{Dbl}\,F)X \longrightarrow_\beta^* F(F(F(FX)))$$

Note the parentheses. If we write without them, left-associativity will enforce another reading: $\textsc{Dbl}\,\textsc{Dbl}\,FX$ is $((\textsc{Dbl}\,\textsc{Dbl})F)X$. In this case, however, the result is the same. Apply the definition of the *outer* Dbl first: it gives $\textsc{Dbl}(\textsc{Dbl}\,F)X$, so again F is applied four times.

5.4 Data as functions

5.4.1 Booleans

What must a Boolean do? It must choose between two alternatives. Define

$$\textsc{True} = \lambda x.\lambda y.x \qquad \textsc{False} = \lambda x.\lambda y.y$$

So True is our old friend K, and False is the dual: "Take the first argument, forget it, take the second and return it".

Then $\textsc{True}\ AB \longrightarrow_\beta^* A$ and $\textsc{False}\ AB \longrightarrow_\beta^* B$. A conditional needs no additional mechanism:

$$\textsc{If} = \lambda p.\lambda x.\lambda y.pxy$$

That is, $\textsc{If}\ ABC$ is the λ-way of writing "if A is true, then B, else C". Indeed:

$$\textsc{If}\ \textsc{True}\ BC \longrightarrow_\beta^* \textsc{True}\ BC \longrightarrow_\beta^* B$$

and

$$\textsc{If}\ \textsc{False}\ BC \longrightarrow_\beta^* \textsc{False}\ BC \longrightarrow_\beta^* C$$

To negate a Boolean, exchange its two alternatives:

$$\textsc{Not} = \lambda p.\lambda x.\lambda y.pyx$$

For example,

$$\textsc{Not}\ \textsc{True} \longrightarrow_\beta \lambda x.\lambda y.\textsc{True}\ yx \longrightarrow_\beta^* \lambda x.\lambda y.y = \textsc{False}$$

Conjunction can be implemented by

$$\textsc{And} = \lambda p.\lambda q.pqp$$

That is, if p is true, return q; otherwise return p, which is already false.

5.4.2 Natural numbers

If the only thing we can do is apply functions, how can we simulate a number $n \in \mathbb{N}$? Well, the number n can tell us to apply a function n times. Define the **Church numerals**

$$\overline{0} = \lambda f.\lambda x.x \qquad \overline{1} = \lambda f.\lambda x.fx \qquad \overline{2} = \lambda f.\lambda x.f(fx) \qquad \ldots$$

In general, $\overline{n} = \lambda f.\lambda x.f^n(x)$, where $f^0(x) = x$ and $f^{n+1}(x) = f(f^n(x))$. The superscript here is an abbreviation for repeated application. In particular, $\textsc{Dbl}$ from Example 35 is $\overline{2}$.

To obtain $n + 1$, the successor of n, perform its repetitions and apply f once more. That is, $\overline{n+1}$ can be represented as $\lambda f.\lambda x.f(\overline{n}fx)$. Literally: "take f, take x, apply f to x exactly n times, and then once more".

So the function that takes $\overline{n}$ and returns $\overline{n+1}$ is then:

$$\textsc{Succ} = \lambda n.\lambda f.\lambda x.f(nfx)$$

For example,

$$\textsc{Succ}\,\overline{2} \longrightarrow_\beta^* \lambda f.\lambda x.f(f(fx)) = \overline{3}$$

To add m and n, apply successor m times, starting at $\overline{n}$:

$$\textsc{Add} = \lambda m.\lambda n.m\,\textsc{Succ}\,n$$

A Church numeral can iterate any function, including a function on Church numerals. Here $\overline{m}$ receives SUCC as the function to repeat and $\overline{n}$ as the starting value. For example,

$$\text{ADD}\,\overline{2}\,\overline{3} \longrightarrow_\beta^* \text{SUCC}(\text{SUCC}\,\overline{3}) \longrightarrow_\beta^* \overline{5}.$$

For multiplication, add n a total of m times, starting at zero:

$$\text{MULT} = \lambda m.\lambda n.m\,(\text{ADD}\,n)\,\overline{0}$$

Note the parentheses! ADD n is a function that takes a numeral and adds n to it. Repeating this function m times from $\overline{0}$ gives $\overline{n}, \overline{2n}, \ldots, \overline{mn}$; if $m = 0$, the result is simply $\overline{0}$.

Remark 36. Note that $\overline{0}$ and FALSE are the same term up to parameter names.

5.5 Halting and nontermination

What would be the λ-counterpart of our `never_stop` program? Of the Turing machine that runs forever to the right?

A term is in **normal form** if it contains no subexpression of the form $(\lambda x.M)N$. For example, $\lambda x.x$ and all Church numerals are normal forms. A term *has* a normal form if some finite sequence of reductions reaches one.

Let

$$\Delta = \lambda x.xx \qquad \Omega = \Delta\Delta$$

Then $\Omega \longrightarrow_\beta \Omega \longrightarrow_\beta \cdots$. Its only possible step reproduces the same term, so it has no normal form. Notice that Δ itself is a normal form.

Remark 37. Compare Ω to the 'diagonal' program δ of Theorem 7. The former takes the input and applies it to itself. The latter checked whether the input program halts on its own code.

Different subexpressions may be ready to reduce:

$$(\lambda x.\text{ID})\,\Omega \longrightarrow_\beta \text{ID} \qquad \text{and} \qquad (\lambda x.\text{ID})\,\Omega \longrightarrow_\beta (\lambda x.\text{ID})\,\Omega$$

That is, reducing the outer application gives ID immediately. Reducing inside the argument can continue forever. Thus, the existence of an infinite reduction does *not* prove that a term has no normal form.

Theorem 38 (Church–Rosser)**.** *If $M \longrightarrow_\beta^* P$ and $M \longrightarrow_\beta^* Q$, then there is a term R such that $P \longrightarrow_\beta^* R$ and $Q \longrightarrow_\beta^* R$, allowing renaming of bound variables (alpha conversions).*

This property is called **confluence**. In particular, a term cannot have two different normal forms: neither could reduce further to a common result. Confluence does not say that every choice of reductions terminates. There is, however, a strategy for choosing what to reduct first.

Theorem 39 (Normalisation)**.** *If a term has a normal form, repeatedly reducing the leftmost outermost reducible application reaches it.*

'Outermost' means that the application is not inside another reducible application; among those, take the leftmost one.

5.6 Trying it in Scheme

Scheme is a *functional programming language* in which functions can be passed around just as in our examples. An online interpreter, such as `https://try.scheme.org/`, lets us try:

```
(define  dbl (lambda (f) (lambda (x) (f (f x)))))
(define  succ (lambda (x) (+ x 1)))

(( dbl  succ) 10)           ; 12
((( dbl  dbl)  succ) 10)    ; 14
```

Here `define` names a value, `(f x)` applies it, and semicolons introduce comments. Scheme supplies built-in numbers and evaluates arguments before function calls.

Lecture 6. More lambda calculus

We have represented Booleans and numbers by lambda terms. We now turn to *data structures*: pairs and lists. To work with lists, we will need *recursion*. Fasten your seatbelts.

6.1 Pairs

To store two values, make a function that passes both to whoever wants to use them:

$$\textsc{Pair} = \lambda a.\lambda b.\lambda f.f\,a\,b \qquad \textsc{Fst} = \lambda p.p\,\textsc{True} \qquad \textsc{Snd} = \lambda p.p\,\textsc{False}$$

Thus $\textsc{Fst}\,(\textsc{Pair}\,A\,B) \longrightarrow_\beta^* A$, and similarly for the second component.

Embrace the λ-philosophy: a pair is a function; to get its first or second value, *apply it* to the right argument. If you want a sum of the two components, apply your pair to the function of addition: $(\lambda f.f\,a\,b)\,\textsc{Add} \longrightarrow_\beta \textsc{Add}\,a\,b$.

Example 40. We use pairs to compute the predecessor function (i.e. $\max\{x-1,0\}$) for Church numerals. Keep the previous and current counter values in a pair. Starting from $(0,0)$, the update $(\overline{m},\overline{n}) \mapsto (\overline{n},\overline{n+1})$ gives $(0,1),(1,2),(2,3),\ldots$. It is implemented by

$$\textsc{Upd} = \lambda p.\textsc{Pair}\,(\textsc{Snd}\,p)\,(\textsc{Succ}(\textsc{Snd}\,p)).$$

After n updates, the first component is $\max\{n-1,0\}$, so

$$\textsc{Pred} = \lambda n.\textsc{Fst}(n\,\textsc{Upd}\,(\textsc{Pair}\,\overline{0}\,\overline{0})).$$

We can now implement *truncated subtraction*: $m \dot{-} n = \max\{m-n,0\}$. Apply predecessor n times to m. Once zero is reached, it stays zero:

$$\textsc{Subt} = \lambda m.\lambda n.n\,\textsc{Pred}\,m$$

6.2 Recursion in lambda calculus

In Python, we can define a function that calls itself:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

In pure lambda calculus, a term is a *finite syntactic expression*. Writing $\textsc{True} = \lambda x.\lambda y.x$ is just a naming convention: "wherever you see True, substitute $\lambda x.\lambda y.x$". We cannot define a term literally as, say, $A = \lambda x.Ax$. It's like saying that inside the text of 'The Three Musketeers' there is a chapter containing

exactly the text of the full 'The Three Musketeers'.[10] An equation describing how a term *computes*, however, is a different matter.

Two terms A and B are **β-equal**, written $A =_\beta B$, if one can pass from one to the other by finitely many beta reductions, reversed beta reductions, and alpha conversion. By Theorem 38, two β-equal terms either both lack a normal form or have the same normal form, up to alpha conversion. This remains true when β-equal terms are placed in the same surrounding term, so either can replace the other in a program without changing its normal-form result. Their reduction sequences, however, need not be the same.

Suppose we want a term FACT implementing factorial. Its body must test for zero and, in the other case, use FACT on the predecessor. First write this body with a parameter f in place of the recursive function:

$$G = \lambda f.\lambda n.\text{IF}\,(\text{ISZERO}\,n)\,\overline{1}\,(\text{MULT}\,n\,(f\,(\text{PRED}\,n))).$$

Here ISZERO is the zero test from Exercise sheet 5, and PRED is the predecessor function from Example 40. Note that $G\,f$ performs one layer of the computation, delegating the smaller factorial to the function f that it takes as a parameter. It is not yet factorial, because that function still has to be supplied.

We want the resulting function to use *itself* for the delegated calls. Thus we seek a term FACT satisfying

$$\text{FACT} =_\beta G\,\text{FACT}.$$

Such a term FACT is called a **fixpoint**[11] of G. The goal is to find a term satisfying this equation for our chosen body G.

Theorem 41 (Fixpoint theorem)**.** *There is a closed term Θ such that, for every term F,*

$$\Theta F \longrightarrow_\beta^* F(\Theta F).$$

Proof. Put $U = \lambda u.\lambda f.f(uuf)$ and $\Theta = UU$. Then

$$\Theta F = UUF \longrightarrow_\beta (\lambda f.f(UUf))F \longrightarrow_\beta F(UUF) = F(\Theta F). \qquad \square$$

Set $\text{FACT} = \Theta G$. Unfolding FACT gives $G\,\text{FACT}$: the parameter f in the body is replaced by FACT itself. For factorial, this gives

$$\text{FACT}\,\overline{0} \longrightarrow_\beta^* \overline{1}, \qquad \text{FACT}\,\overline{3} \longrightarrow_\beta^* \text{MULT}\,\overline{3}\,(\text{FACT}\,\overline{2}).$$

Reducing further similarly reaches $\text{FACT}\,\overline{1}$ and then $\text{FACT}\,\overline{0}$, which is the base of recursion. We therefore obtain $\overline{6}$. Reduction produces each new call as needed; after expanding the abbreviations, ΘG itself is still a finite term.

Remark 42. Theorem 41 finds a fixpoint of a function F among all lambda terms, not necessarily the terms that F was 'supposed to' work on. For example, $\Theta\,\text{SUCC}$, the fixpoint of the successor function, cannot be a Church numeral: SUCC sends every numeral to a different one.

[10] Formally, let $|B|$ denote the length of a term B, i.e. the number of symbols used to write it down. Literal equality would give $|A| = |\lambda x.Ax| > |A|$, which is impossible.

[11] A grammatically correct way of writing is 'fixed-point', while 'fixpoint' is slang in theoretical computer science.

6.3 Scott lists

To inspect and modify sequences, we will use Scott lists (named after Dana Scott, Church's student and a star theoretician himself), which are different from Church lists from Exercise sheet 6. A Church list supplies the result of folding its tail; a Scott list supplies the tail itself, which is convenient for inspecting a list one element at a time.

A Scott list has two possibilities: it is empty, or it consists of a head and a tail. Encode these alternatives as

$$\textsc{Nil} = \lambda z.\lambda c.z, \qquad \textsc{Cons} = \lambda h.\lambda t.\lambda z.\lambda c.c\,h\,t.$$

Here Cons stands for 'constructor': given a head (an element) and a tail (a list), it constructs a new list, with the given head followed by the given tail.

Given an empty-case value Z and a nonempty-case function C, a list L behaves as follows:

$$\textsc{Nil}\ Z\ C \longrightarrow_\beta^* Z, \qquad (\textsc{Cons}\ H\ T)\ Z\ C \longrightarrow_\beta^* C\ H\ T.$$

A nonempty list passes its head and its *actual tail* to the selected function. For example,

$$\textsc{Tail} = \lambda l.l\ \textsc{Nil}\ (\lambda h.\lambda t.t)$$

returns the tail of a nonempty list, and Nil on an empty list.

Appending one list to another

We want concatenation to satisfy:

$$\textsc{App}\ \textsc{Nil}\ b =_\beta b, \qquad \textsc{App}\ (\textsc{Cons}\ h\ t)\ b =_\beta \textsc{Cons}\ h\ (\textsc{App}\ t\ b).$$

The first equation handles the empty list. The second keeps the head and concatenates the tail with b. Turn this description into a function of the recursive call:

$$F = \lambda f.\lambda l_1.\lambda l_2.l_1\ l_2\ (\lambda h.\lambda t.\textsc{Cons}\ h\ (f\ t\ l_2)), \qquad \textsc{App} = \Theta F.$$

Unfolding the fixpoint once gives

$$\textsc{App}\ l_1\ l_2 \longrightarrow_\beta^* l_1\ l_2\ (\lambda h.\lambda t.\textsc{Cons}\ h\ (\textsc{App}\ t\ l_2)).$$

What is going on here? The first list, supplied for l_1, is itself a case selector: if $l_1 = \textsc{Nil}$, it returns the empty-case value l_2; if $l_1 = \textsc{Cons}\ h\ t$, it calls the second argument with h and t, giving $\textsc{Cons}\ h\ (\textsc{App}\ t\ l_2)$. No separate emptiness test is needed.

Example 43. Write $[A, B]$ for the Scott list with elements A, B, and [] for Nil. Let us concatenate $[A, B]$ and $[C]$:

$$\begin{aligned}\textsc{App}\ [A, B]\ [C] &\longrightarrow_\beta^* \textsc{Cons}\ A\ (\textsc{App}\ [B]\ [C]) \\ &\longrightarrow_\beta^* \textsc{Cons}\ A\ (\textsc{Cons}\ B\ (\textsc{App}\ []\ [C])) \\ &\longrightarrow_\beta^* [A, B, C].\end{aligned}$$

Each recursive call shortens the first list, so the computation terminates on finite lists (unless a list itself contains a non-terminating element such as Ω).

6.4 Computing functions on words

Lambda calculus is Turing-complete: any function $\Sigma^* \rightharpoonup \Sigma^*$ computable by a Turing machine can be computed by a lambda term. This statement may not be clear yet—in lambda calculus, we apply terms to other terms. So how can a term 'compute' a function on words?

For example, we can encode every word $w = a_1 \dots a_n$ by the normal-form Scott-list encoding $L_w = [A_1, \dots, A_n]$, where the terms A_i encode the letters a_i, in some way.[12] Then, for every computable partial function $f : \Sigma^* \rightharpoonup \Sigma^*$ there exists a term F such that:

1. if $f(w) = u$, then $F\, L_w \longrightarrow_\beta^* L_u$;
2. if $f(w)$ is undefined, then $F\, L_w$ does not have a normal form.

To have an idea of how this is possible, recall that Markov normal algorithms can already simulate Turing machines (Exercise sheet 4). To simulate a step of a given normal algorithm on a list L_w, inspect it one element at a time, choose the *first applicable rule*, and replace its *leftmost occurrence*. Recursive functions on lists can compare symbols, search for the pattern, and rebuild the word with the replacement, using concatenation as above. The fixpoint operator lets us repeat these steps, restarting the rule search after each replacement. A terminal rule stops immediately; if no rule applies, return the current word. We omit the full construction.

6.5 Scheme again

In Scheme, the recursive equations for concatenation can be written directly:

```
(define join
  (lambda (xs ys)
    (if (null? xs)
        ys
        (cons (car xs) (join (cdr xs) ys)))))

(join '(a b) '(c))   ; (a b c)
```

Here `car`, `cdr`, and `cons` give the head, tail, and list constructor. The quote marks list data; `'()` is empty. Scheme supplies recursive definitions directly and allows several arguments in one call.

[12] Say, if $\Sigma = \{b_1, \dots, b_m\}$ has m letters, the term for the letter b_i could be $\lambda x_1 \dots \lambda x_m . x_i$, the *selector* of the ith argument.

Lecture 7. Time complexity, classes P and EXP

We now turn from "what can computers do?" to "what can computers do efficiently?" A program that takes a thousand years to finish is of little practical use. How fast can an algorithm be? This is a question for computational complexity theory.

7.1 Running time of an algorithm

The running time of an algorithm usually depends on the size of its input. Finding shortest paths in a large graph, for example, may take longer than in a small one. Here is a more direct example: if a Turing machine working over unary alphabet takes a natural number n and must output 2^n, it needs at least 2^n steps just to write down the answer.

To measure efficiency, we therefore ask *how running time grows with input size*.

In our formalisation, inputs are words, and the size of an input word w is its length, denoted $|w|$. The empty word ε has length 0. Representation matters: the number 5 takes one digit in decimal, three bits in binary (101), and five symbols in unary (11111). Still, any fixed finite input alphabet Σ can be encoded in binary using $\max\{1, \lceil \log_2 |\Sigma| \rceil\}$ bits per symbol. This increases word length by at most a constant factor.

For precise definitions, we fix a model of computation: multitape Turing machines (see Exercise 3.6). For simplicity, we focus on decision problems, or deciding languages. A decider has two halting states, accepting and rejecting, and reaches one of them on every input (see Section Lecture 3.).

Definition 44. *Let $f : \mathbb{N} \to \mathbb{N}$ be a function. A Turing machine M* runs in time $O(f(n))$ *if it halts on every input and there are constants $c > 0$ and k such that, for every input word w with $|w| \geq k$, M halts within $c \cdot f(|w|)$ steps.*

This is a worst-case bound for a particular algorithm represented by M. The notation $O(f(n))$ ignores a constant factor and finitely many small inputs. That is useful when comparing growth rates, but the constant may matter greatly in practice: $10^{10}n$ is still $O(n)$.

Complexity theory also asks how difficult a computational task is, regardless of the algorithm chosen to solve it. We start with the following definition for decision problems.

Definition 45. *A language $L \subseteq \Sigma^*$ is* decidable in time $O(f(n))$ *if there exists a Turing machine that decides L and runs in time $O(f(n))$.*

We can now classify languages by how fast they can be decided. For every $f : \mathbb{N} \to \mathbb{N}$, we obtain a *complexity class*:

$$\mathrm{DTIME}(f(n)) = \{L \subseteq \Sigma^* \mid L \text{ is decidable in time } O(f(n))\}$$

Example 46. Class $\mathrm{DTIME}(1)$ comprises languages decidable in constant time. For example, $\emptyset \in \mathrm{DTIME}(1)$ (reject immediately) and $\Sigma^* \in \mathrm{DTIME}(1)$ (accept immediately). Here is a less trivial example:

$$\mathrm{REV_EVEN} = \{w \in \{0,1\}^* \mid \text{the reverse of } w \text{ represents an even number in binary}\}$$

A binary number is even precisely when its last bit is 0. Since a Turing machine starts at the first bit of w, it can accept when this bit is 0 (or when $w = \varepsilon$) and reject when it is 1. It need not scan the rest of w, so this really is a constant-time decision procedure.

Example 47. Class DTIME (n) contains languages decidable in linear time. For example,

$$\text{PARITY} = \{w \in \{0,1\}^* \mid w \text{ contains an even number of 1s}\}$$

A Turing machine scans the word once, keeping the parity of the number of 1s in its state. Here is another example:

$$\text{EVEN} = \{w \in \{0,1\}^* \mid w \text{ represents an even number in binary}\}$$

A machine decides this by scanning to the end and checking the last bit.

7.2 Polynomial and exponential time

For the language EVEN from Example 47, one can prove that EVEN $\notin$ DTIME (1): a machine that runs in constant time cannot reach the last bit of a sufficiently long word. This proof depends on the model. A computer with random-access memory, given the input length, could jump to the last bit. Exact time classes such as DTIME (1) are therefore sensitive to the underlying model.

Lower bounds for familiar problems are usually much harder to prove. A slow algorithm tells us only how long *that* algorithm takes. To show that a language has no faster algorithm, we must rule out every possible algorithm, including ones we have not discovered.

We therefore consider larger classes that give a more robust view of running time. The class P contains languages decidable in polynomial time; EXP contains those decidable in exponential time:

$$\text{P} = \bigcup_{k \geq 0} \text{DTIME}\left(n^k\right), \qquad \text{EXP} = \bigcup_{k \geq 0} \text{DTIME}\left(2^{n^k}\right).$$

These larger classes smooth over many differences between reasonable models of computation. A multitape Turing machine can simulate a standard computer program with at most polynomial overhead, provided we count the bit cost of operations on large numbers; the converse also holds.

Example 48. Consider the language

$$\text{PALINDROME} = \{w \in \{0,1\}^* \mid w \text{ reads the same forwards and backwards}\}.$$

A one-tape machine can repeatedly compare and cross off the leftmost and rightmost unmatched symbols. Each round scans at most $O(n)$ cells, and there are at most $n/2$ rounds. Thus PALINDROME $\in$ DTIME $\left(n^2\right) \subseteq$ P.

Example 49. In SUBSET-SUM, the input is a list of nonnegative binary integers $a_1, \ldots, a_m$ and a target t. The language is

$$\text{SUBSET-SUM} = \left\{ \langle a_1, \ldots, a_m, t \rangle \;\middle|\; \exists I \subseteq \{1, \ldots, m\} : \sum_{i \in I} a_i = t \right\}.$$

We can test each of the 2^m subsets I. For each one, add the selected numbers and compare the sum with t. These operations take polynomial time in the input length n. Since $m \leq n$, the whole search takes $2^m \operatorname{poly}(n) = 2^{O(n)}$ time. Thus SUBSET-SUM $\in$ EXP.

We should keep in mind that the classes P and EXP hide important details. As noted above, $O(f(n))$ hides a constant factor. The class P, moreover, puts linear-time languages in the same class as those for which only, say, $O(n^{20})$ algorithms are known. We nevertheless treat polynomial time as a theoretical notion of *efficient computation*.

This convention can seem paradoxical. Why is n^{100} 'efficient', while $n^{\log\log\log n}$ is not? The latter function eventually exceeds every polynomial, but its exponent is small on inputs we could realistically store. Polynomial time describes long-run growth, not a guarantee of practical speed.

Still, exponential running time often comes from checking every candidate in a large set (as in Example 49). Polynomial algorithms, on the other hand, usually exploit some 'smart ideas'. It is not rare that an initially high-degree polynomial algorithm is improved over time by refining the underlying idea.

7.3 Time hierarchy

Example 49 gave an upper bound on the complexity of SUBSET-SUM. Can we do better? Or can we prove that there is no polynomial-time algorithm for this problem, i.e. that SUBSET-SUM $\notin$ P? Actually, do we know *any* language outside P? The next theorem says yes, although its proof constructs a rather artificial example. For SUBSET-SUM, membership in P is open.

Theorem 50. *For all $k \geqslant 0$,* $\mathrm{DTIME}\left(n^k\right) \subset \mathrm{DTIME}\left(n^{k+1}\right)$. *Moreover,* $\mathrm{P} \subset \mathrm{EXP}$.

Proof. The non-strict inclusions are immediate: for every fixed k, $n^k = O(n^{k+1})$ and $n^k = O(2^n)$. For $k = 0$, strictness follows from $\mathrm{EVEN} \in \mathrm{DTIME}(n) \setminus \mathrm{DTIME}(1)$. We give the diagonal argument for $\mathrm{DTIME}(n) \subset \mathrm{DTIME}\left(n^2\right)$; the other positive integer exponents work similarly.

Consider words of the form $\langle M\rangle\#0^r$, where $\langle M\rangle$ is an encoding of the Turing machine M suitable for the simulation of M by the universal machine, # is a separator symbol (only given here for visual convenience), and 0^r is a sequence of $r \geq 2$ zeros.

On input $w \in \{0, 1, \#\}^*$, a machine M' rejects if w is not of the form above or if $|\langle M\rangle| > \log r$. Otherwise, M' simulates M on *the whole input* w for at most $\lfloor n^{3/2}\rfloor$ steps, where $n = |w|$. If M halts, M' gives the opposite answer (i.e. rejects if M accepts and accepts if M rejects); if M does not halt within the limit, M' rejects. Let L' be the language decided by M'.

First, $L' \in \mathrm{DTIME}\left(n^2\right)$. This uses a standard fact about universal simulation: a fixed multitape machine can simulate T steps of a machine whose description has $O(\log n)$ bits in $O(n + T(\log n)^c)$ time, for some fixed c, when T is polynomial in n.[13] Parsing w and maintaining the time limit also fit within $O(n^2)$. With $T = \lfloor n^{3/2}\rfloor$, the total is $O(n^2)$.

Second, $L' \notin \mathrm{DTIME}(n)$. Suppose a machine M decided L' in at most cn steps for all sufficiently long inputs. Choose r large enough that $|\langle M\rangle| \leq \log r$ and $cn < \lfloor n^{3/2}\rfloor$ for $w = \langle M\rangle\#0^r$. On this input, M' sees M halt and returns the opposite answer, contradicting the assumption that they decide the same

[13] These details are very technical, but interested readers can consult [Arora & Barak, Computational Complexity: A Modern Approach]

language. For any fixed $k \geq 1$, a limit of $n^{k+1/2}$ steps gives the analogous separation between $\text{DTIME}\left(n^k\right)$ and $\text{DTIME}\left(n^{k+1}\right)$.

The same argument proves $\text{P} \subset \text{EXP}$: simulate $M(w)$ for up to 2^n steps, where $n = |w|$, and give the opposite answer if M halts. Even with the overhead of universal simulation, this machine runs in exponential time, while 2^n eventually exceeds the running time of any polynomial-time M. □

Lecture 8. Class NP

We will use Turing machines that decide languages by halting in one of two states: accepting or rejecting (see Section Lecture 3.).

Definition 51. *A language $L \subseteq \Sigma^*$ is in* NP *if there are a polynomial $p(n)$ and a Turing machine V (the* verifier*), running in polynomial time in the length of its input pair, such that, for every $w \in \Sigma^*$,*

$$w \in L \iff \exists s \in \Sigma^* \text{ such that } |s| \le p(|w|) \text{ and } V \text{ accepts } (w, s).$$

The word s is called a certificate *for w.*

In other words, a word w belongs to a language in NP when its membership can be 'justified' by a not-too-long proof (at most polynomial in the length of w). This proof must also be 'checkable' in polynomial time. It need not be unique: every $w \in L$ has at least one proof, while no $w \notin L$ has a proof accepted by the verifier. We do not require the verifier to find a proof.

Example 52. *Boolean formulas* are built from the constants T (true) and F (false), variables $x_1, x_2, \dots$, and the Boolean operations negation, conjunction, and disjunction. A formula $\varphi(x_1, \dots, x_n)$ is called *satisfiable* if some assignment of Boolean values to its variables makes it true. Thus we have a language:[14]

$$\text{SAT} = \{\varphi \text{ a Boolean formula} \mid \varphi \text{ is satisfiable}\}$$

As usual, we assume that formulas are encoded as words in some alphabet, say $\{T, F, x, 0, 1, \neg, \wedge, \vee, (,)\}$, and thus can be given as input to a Turing machine.

We prove that SAT $\in$ NP. List the variables that occur in φ as $x_1, \dots, x_n$. Let V take a pair (φ, s), where $s \in \{T, F\}^n$ is a word of n letters, and interpret its ith letter as the value assigned to x_i. The machine evaluates φ on this assignment, accepting if it becomes true and rejecting otherwise. It also rejects malformed inputs: for example, φ may not be a well-formed formula, or s may not be a sequence of n Boolean values. Parsing and evaluating the formula take polynomial time. For each satisfiable formula, a satisfying assignment is the required polynomial-size certificate: there are at most $|\varphi|$ distinct variables in the encoded formula. For a non-satisfiable formula, or an input that is not a formula, no certificate is accepted.

Example 53. In a graph, a *clique* is a set of pairwise adjacent nodes. Consider the following language:

$$\text{CLIQUE} = \{\langle G, k\rangle \mid G \text{ is an undirected graph, } k \ge 0, \text{ and } G \text{ has a clique of size at least } k\}$$

Again, CLIQUE $\in$ NP. If the input is malformed, or if $k > |V(G)|$, the verifier rejects immediately. Otherwise, the certificate lists k distinct nodes of G. The verifier checks that every pair in the list is adjacent. Since $k \le |V(G)|$, the certificate has polynomial length, and the checks take polynomial time.

In the two examples above, membership in the language is decided by establishing the existence of something, and this something itself plays the role of the certificate: it is the witness whose existence is at issue. This is a common pattern when talking about class NP, and we will see it again and again. In fact, the usual decision problem associated with a language $L \in$ NP—"decide whether $w \in L$"—is reformulated by the definition of NP as "decide whether there exists a certificate for w that the verifier accepts". This, in turn, connects it to a *search problem*—"find the certificate". Class NP is important because many practical search problems (find this or that in a graph, find an assignment of values to variables that satisfies certain constraints, etc.) can be associated with languages in NP; the answers one is looking for play the role of certificates.

[14] A standard variant, CNF-SAT, restricts the input formulas to conjunctive normal form, see Lecture **??**.

Class NP in relation to classes P and EXP

Proposition 54. P ⊆ NP ⊆ EXP

Proof. Suppose $L \in$ P, so there is a machine M that runs in polynomial time and accepts w if and only if $w \in L$. We construct a verifier V for L: given (w, s), it ignores s and runs M on w. Take $p(n) = 0$, so the only possible certificate is the empty word. Then V accepts a pair exactly when $w \in L$, as required.

Now suppose $L \in$ NP, with verifier V and a polynomial bound $p(n)$ on certificate length. Imagine the following machine M'. Given w, it considers *every* word s of length at most $p(|w|)$ and runs V on (w, s), accepting if and only if V accepts at least one pair. It recognises L: if there is a certificate for w, it will find it in its long search. It also runs in exponential time. Let $q(n)$ be a polynomial upper bound on V's running time. On an input of length $|w| = n$, each run takes at most $q(n + p(n)) = n^{O(1)}$ steps. Since Σ is fixed, there are at most $2^{O(p(n))} = 2^{n^{O(1)}}$ candidate certificates. The enumeration and simulation overhead is polynomial per candidate, so the total running time is still $2^{n^{O(1)}}$. Thus $L \in$ EXP. □

By Theorem 50, P ⊂ EXP, strictly. What about the two inclusions in Proposition 54? Are they strict? It is *widely believed* that they are, but this remains unproved.

Indeed, P is the class of problems that can be solved efficiently; NP is the class of problems for which a proposed yes-answer (a certificate) can be checked efficiently. It is counter-intuitive that being able to check something quickly might mean being able to find it quickly, when the search space can be exponentially large. Yet so far nobody has found a precise mathematical argument that rules out this possibility. This is where the theory of computation starts to feel less like mathematics and more like science: we hypothesise certain 'laws of computation', but have not proved them mathematically.

In 2000, the Clay Mathematics Institute selected seven long-standing, difficult problems in mathematics as the Millennium Prize Problems. Its total prize fund was US7$million, $withUS1 million allocated for a solution to each problem; the P-versus-NP question is one of them.